Lab 2

MSDS684_S70_Reinforcement Learning

Maryi Tatiana Palacios Giraldo

Regis University

March 2026

Project Overview

This lab investigates the use of Dynamic Programming (DP) methods to solve Markov Decision Processes (MDPs), focusing on Policy Iteration and Value Iteration as presented in Sutton & Barto (Chapter 4). The objective is to understand how full knowledge of the environment's transition dynamics can be used to compute optimal policies and value functions.

The core problem explored in this lab is how an agent can determine the optimal sequence of actions in a discrete environment to maximize cumulative rewards. Unlike simpler reinforcement learning settings such as multi-armed bandits, this problem involves state transitions, making it a full MDP. The lab specifically examines how different DP algorithms converge to optimal solutions and how their performance varies depending on environmental characteristics such as determinism and stochasticity.

The custom environment implemented is a GridWorld of size 4×4, where each state represents a position in the grid. The state space is discrete with 16 states, and the action space consists of four possible actions: up, down, left, and right. The reward structure assigns a penalty of -1 for each step, encouraging the agent to reach a terminal state in the fewest possible steps. Two terminal states are defined at the corners of the grid, and the episode terminates when one of these states is reached.

To further explore the impact of uncertainty, both deterministic and stochastic versions of the environment are considered. In the stochastic setting, actions do not always lead to the intended outcome; instead, there is an 80% probability of moving in the intended direction and a 10% probability of moving in each perpendicular direction. This introduces uncertainty in the transition dynamics and makes planning more complex.

The lab also applies these algorithms to the FrozenLake environment from Gymnasium, which features highly stochastic transitions and sparse rewards. This provides a comparison between a controlled custom environment and a more complex, standard benchmark.

It is expected that both Policy Iteration and Value Iteration will converge to the same optimal policy, as guaranteed by DP theory. However, Value Iteration is hypothesized to converge faster due to its direct use of the Bellman optimality equation, while Policy Iteration may require more computation per iteration due to the policy evaluation step. Additionally, stochastic environments are expected to slow convergence due to increased uncertainty in state transitions.

Deliverables

GitHub Repository:

https://github.com/MPalacios-Analytics/DataScience_Projects/tree/5ab1b4c1aa76d0cd11f11609b9f154a9ddf27ccd/06_Reinforcement_Learning/Lab_2

Implementation Summary

This lab implements Dynamic Programming algorithms including Policy Evaluation, Policy Iteration, and Value Iteration using NumPy arrays. A custom GridWorld environment was developed with configurable stochastic transitions and a full transition model $P(s', r \mid s, a)$. Both in-place and synchronous versions of policy evaluation were implemented to analyze convergence behavior. Experiments were conducted on deterministic and stochastic environments, as well as on Gymnasium's FrozenLake-v1 environment. Performance was evaluated in terms of convergence speed (wall-clock time) and resulting value functions and policies.

Key Results & Analysis

The results show that both Policy Iteration and Value Iteration successfully converge to the same optimal value function and policy across all environments tested. This is consistent with the theoretical guarantees of Dynamic Programming, as both algorithms ultimately solve the Bellman optimality equation (Sutton & Barto, Chapter 4).

However, the convergence speed differs significantly between the two methods. Value Iteration consistently converges faster than Policy Iteration in both deterministic and stochastic environments. This is because Value Iteration updates the value function directly using the Bellman optimality update, whereas Policy Iteration requires a full policy evaluation step at each iteration, which is computationally more expensive.

Additionally, the stochastic version of the GridWorld environment results in slower convergence compared to the deterministic version. The presence of probabilistic transitions introduces additional complexity, as each action must consider multiple possible outcomes. This aligns with theoretical expectations, as uncertainty increases the difficulty of accurately estimating value functions.

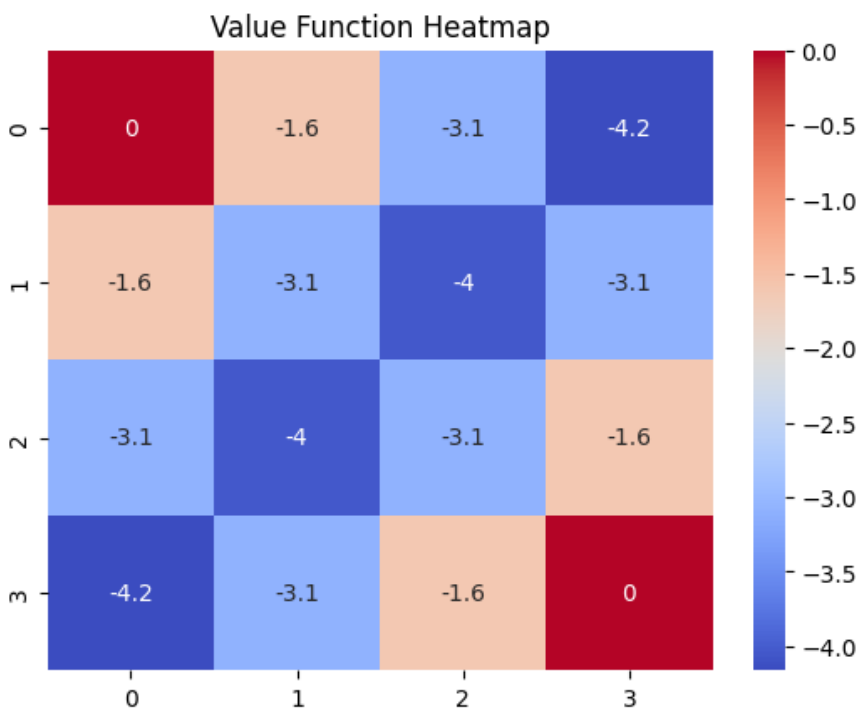
Visualizations of the value function using heatmaps reveal a clear gradient toward terminal states in the GridWorld environment, indicating that the agent learns to minimize cumulative penalties by reaching terminal states as efficiently as possible. In

contrast, the FrozenLake environment produces less structured value functions due to its sparse reward structure and high stochasticity.

The optimal policies also differ significantly between environments. In GridWorld, the policy is direct and follows the shortest path to the goal. In FrozenLake, the policy is less intuitive and more conservative, reflecting the need to account for the risk of unintended transitions. This demonstrates how environmental dynamics, particularly stochasticity and reward sparsity, can significantly influence optimal decision-making strategies.

Visualizations

Figure 1: Heatmap GridWorld



The value function shows a clear gradient toward the terminal states, indicating that the agent minimizes cumulative penalties by following the shortest path.

Figure 2: GridWorld Optimal Policy

↑ ← ← ←

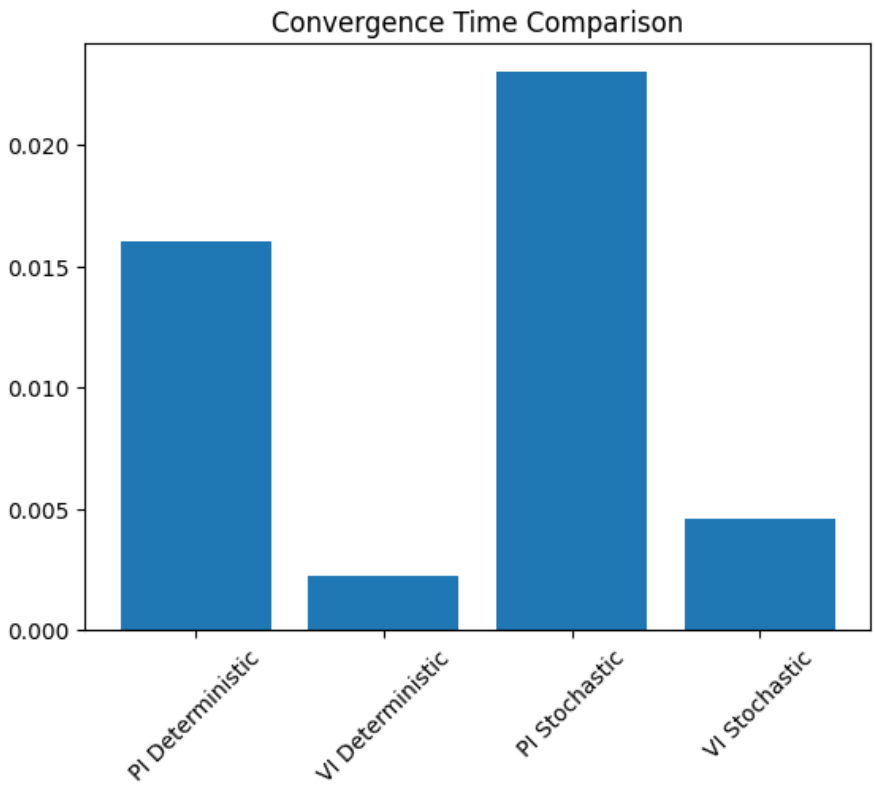
↑ ↑ ← ↓

↑ ↑ ↓ ↓

↑ → → ↑

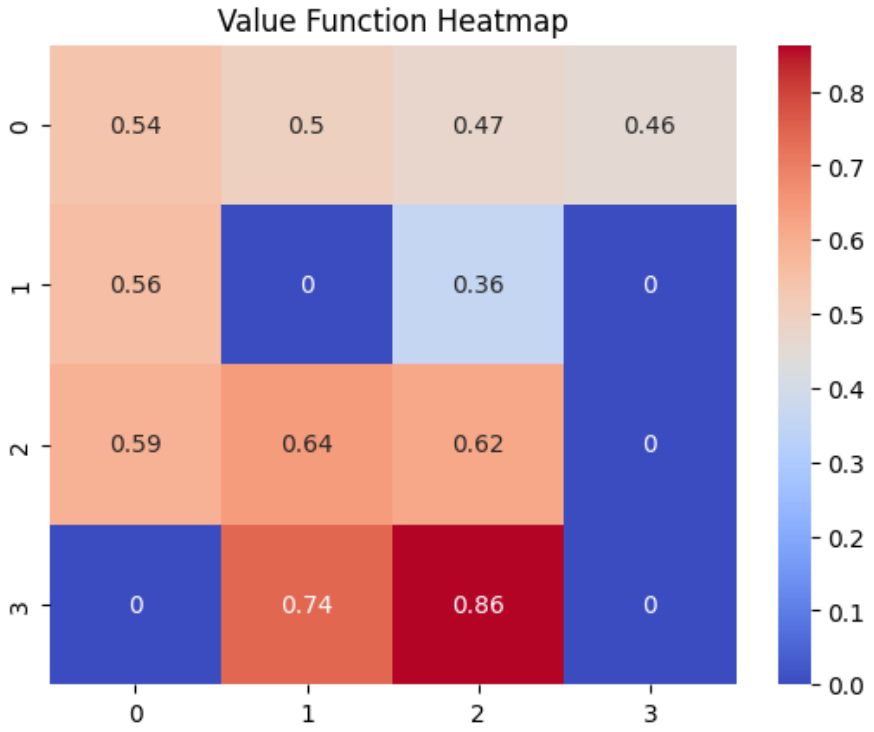
The optimal policy directs the agent toward the terminal states using the most efficient path. The arrows indicate the best action at each state, forming a strategy that minimizes cumulative penalties.

Figure 3: Convergence Plot:



Value Iteration converges significantly faster than Policy Iteration in both deterministic and stochastic environments. Stochastic environments increase convergence time due to uncertainty in transitions.

Figure 4: FrozenLake heatmap



The FrozenLake value function is less structured due to stochastic transitions and sparse rewards, illustrating increased uncertainty in the environment.

AI Use Reflection

Initial Interaction

I used ChatGPT to generate an initial implementation of the GridWorld environment and Dynamic Programming algorithms.

I asked:

"How can I implement policy evaluation for a GridWorld environment using NumPy?"

The AI suggested using an iterative update over all states, computing the expected value using the Bellman expectation equation until convergence.

Iteration Cycle

During development, I encountered several issues that required iterative debugging.

In the first iteration, I faced a ModuleNotFoundError when trying to import Gymnasium. I resolved this by installing the package using pip. This was a straightforward environment issue.

In a second iteration, I encountered import errors when organizing my code into folders.

I asked:

"Why is Python not finding my module even though it is in the same project folder?"

The AI suggested modifying the Python path using sys.path.append() or restructuring the project. I initially followed this suggestion, but it made the code harder to maintain. Instead, I simplified the folder structure, which resolved the issue more cleanly.

In a third iteration, I obtained incorrect value outputs when implementing Policy Iteration.

I asked:

"Why is my policy iteration not converging to the correct values?"

The AI suggested that the issue could be in the policy update step. It recommended ensuring that the policy selects the best action deterministically using a one-hot encoding. After applying this change, the value function converged correctly and matched expected results.

Critical Evaluation

While ChatGPT provided useful guidance, not all suggestions worked optimally.

For example, modifying the Python path solved the import issue temporarily but introduced unnecessary complexity. I determined this was not ideal by observing that simpler project structures avoided the issue entirely.

This showed that AI suggestions may be technically correct but not always the best design choice.

Learning Reflection

Through this process, I developed a deeper understanding of Dynamic Programming algorithms, particularly how policy iteration alternates between evaluation and improvement steps.

The debugging process helped me understand that incorrect policy updates can prevent convergence, reinforcing the importance of correctly implementing the greedy policy improvement step.

I also learned that AI tools are most effective when used iteratively and critically evaluated, rather than followed blindly.

Speaker Notes

• Problem: Solving MDPs using Dynamic Programming in GridWorld and FrozenLake
• Method: Implemented Policy Iteration and Value Iteration with full transition model
• Design choice: Added stochastic transitions to analyze uncertainty
• Key result: Value Iteration converges faster than Policy Iteration
• Insight: Stochastic environments slow convergence and alter optimal policies
• Challenge: Debugging transition model and policy updates
• Connection: These DP methods form the foundation for model-free RL in later weeks